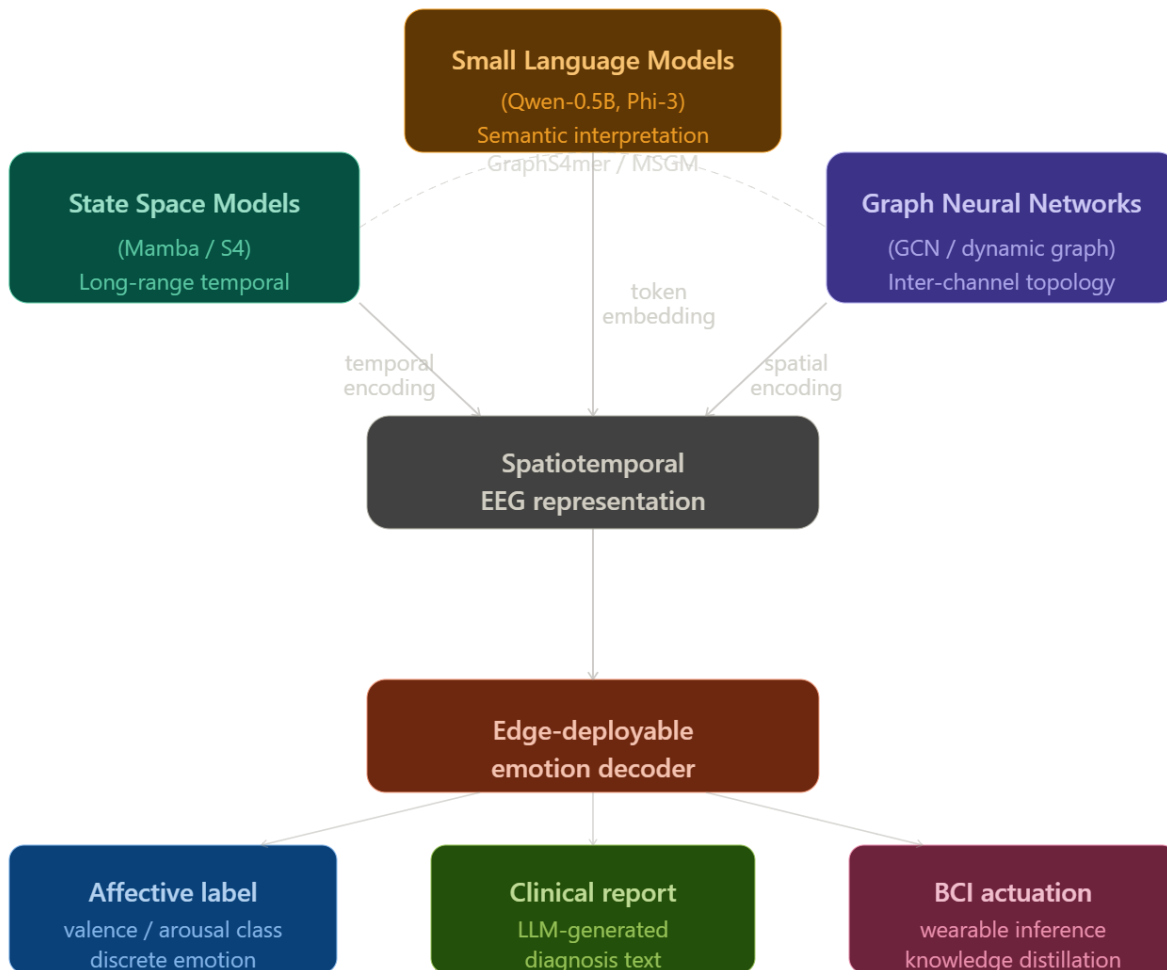




The core problem each family solves

The temporal problem: EEG is a long sequence. Emotional state doesn't resolve in a single window — it unfolds over seconds, with dependencies across timescales (delta to gamma bands). Transformers handle this with quadratic attention complexity, which is prohibitive at the edge. SSMs, specifically Mamba and S4, offer superior scalability over Transformers, especially for long sequences, via selective state spaces for linear-time sequence modeling.

The spatial problem: EEG electrodes are not independent sensors — they sit on a non-Euclidean manifold. Brain functional connectivity (which channels co-activate during specific emotions) is the actual signal. Standard CNNs and RNNs treat channels as isolated time series, discarding this topology entirely. GNNs have emerged as the dominant framework to address these spatial limitations, characterizing EEG channels as nodes within a topological graph.

Where GNN + SSM fuse: the landmark architecture

The foundational paper combining these two families is **GraphS4mer** (Tang et al., CHIL 2023). The key insight is representing multivariate biosignals as time-dependent graphs. GraphS4mer uses the Structured State Space architecture (S4) to capture long-range temporal dependencies in biosignals, and adds a graph structure learning layer to learn dynamically evolving graph structures in the data. This is important: the graph topology is not fixed (by anatomical priors) but *learned as a function of the signal*, meaning the connectivity map adapts per emotional state.

The 2026 paper **MSGM** takes this further specifically for emotion. MSGM integrates multi-window temporal segmentation, bimodal spatial graph modeling, and efficient fusion via the Mamba architecture. By segmenting EEG signals across diverse temporal scales and constructing global-local graphs with neuroanatomical priors, it captures fine-grained emotional fluctuations and hierarchical brain connectivity at linear complexity. On the SEED dataset under subject-independent protocols, MSGM achieves 83.43% accuracy and 85.03% F1 score, and confirms suitability for real-time applications by achieving millisecond-level inference (151 ms) on the NVIDIA Jetson Xavier NX edge device.

This is the GNN–SSM synergy crystallized: GCN handles the *where* (which brain regions are talking), Mamba handles the *when* (how that conversation evolves), and the combination runs at edge-viable latency.

Where SLMs enter

SLMs play a qualitatively different role — they operate *above* the signal decoding layer, not inside it. The clearest example is **EEG Emotion Copilot**. This system uses a lighter LLM in a local server to perform multiple tasks using EEG signals, including emotion recognition, custom diagnostics, treatment recommendations, and automatic creation of medical records. It employs strong privacy safety measures through novel data processing protocols. The Qwen2-0.5B pre-trained model is used for pruning, achieving a 50% reduction over model parameters. A warm-up during the fine-tuning phase using LoRA gradually increases the learning rate.

The practical logic here is important: lightweight LLMs are optimally run locally to safeguard participant privacy and data security. For specific local tasks, complex DNNs are often excessively redundant, so effective model pruning is essential to achieve a lightweight language model suitable for local execution.

The SLM's job is to convert a structured affective label (produced by the GNN+SSM backbone) into natural-language output — a clinical note, a user-facing explanation, a diagnostic summary.

This is a genuine division of labor: the GNN+SSM stack is the signal processor; the SLM is the semantic surface.

Why this matters for your architecture specifically

Your EEG-as-teacher, wearable-as-student architecture maps very cleanly onto this convergence:

During EEG calibration (teacher phase): A GNN+SSM backbone (like MSGM or GraphS4mer) is the appropriate EEG decoder. GNNs can encode the subject's individual functional connectivity graph; SSMs capture the temporal dynamics of emotional state transitions. The output is a rich affective embedding.

During wearable inference (student phase): A distilled model running on ECG/HRV/EDA at the edge is the inference path. The GNN+SSM teacher provides the supervision signal for the student model, and a quantized SLM (Qwen2-0.5B with LoRA, for instance) provides the interpretability layer that generates natural language from the student's discrete outputs.

The convergence point that doesn't yet exist at scale: A unified architecture where the graph structure (functional connectivity) is jointly learned from EEG during calibration *and* transferred to the wearable sensor graph (e.g., ECG lead topology). Dynamic GNNs that learn evolving brain graphs during calibration could, in principle, inform a simpler static graph for the wrist sensor student — this is a research gap worth noting.

State of the field in brief

The GNN + SSM integration is now well-validated on SEED, DEAP, and FACED benchmarks, and is showing up in the form of edge-deployable architectures (Jetson Xavier, Raspberry Pi-class devices). A parallel line of work proposes a novel state space model with dynamic graph neural network specifically for EEG event detection. SLM integration for EEG is newer and mostly focused on downstream generation tasks (medical records, diagnostics) rather than signal decoding, meaning the SLM is almost always the terminal output layer rather than a component in the decoding pipeline itself.

The open architectural frontier is a tightly coupled pipeline where GNN topology learning, SSM temporal modeling, and SLM output generation are jointly fine-tuned end-to-end on EEG — right now they are typically trained in stages. That's where the most interesting results are likely to appear in 2026–27.

The core idea: LLM-to-SLM knowledge distillation

Knowledge distillation (KD) in the language model context means training a smaller "student" model to replicate the behavior of a larger "teacher" — not just matching final predictions, but learning from the teacher's internal representations, probability distributions, and reasoning patterns.

There are three main distillation axes you can pull:

1. Output-level (black-box) distillation The student learns from the teacher's output token probability distributions (logits), not just the hard label. This is the classic Hinton KD formulation. The teacher's "soft targets" encode uncertainty and inter-class relationships that hard labels discard. For the EEG→affective label→text pipeline, this means the SLM learns not just "this is sadness" but the teacher's full distribution over emotion categories and their linguistic renderings.

2. Feature-level (white-box) distillation The student learns to match intermediate representations from the teacher — hidden states, attention maps, or layer activations. This is harder to implement (requires architectural alignment) but produces much better small models. Techniques like **MiniLM**, **TinyBERT**, and **DistilBERT** are all feature-level distillation products.

3. Reasoning-level (chain-of-thought) distillation The most recent and most relevant approach for your use case. The teacher LLM generates reasoning chains — step-by-step explanations — and the SLM is trained to produce the same chain, not just the final answer. Papers like **Alpaca**, **Vicuna**, and more recently **MedDistill** variants show that small models can learn surprisingly complex reasoning behaviors this way. For EEG emotion copilot-style tasks (generate a clinical note from an affective label), this is the right distillation target.

What this looks like concretely for EEG emotion interpretation

Your pipeline has a specific distillation structure:

```
EEG signal
  ↓
GNN+SSM backbone → affective embedding + emotion label
  ↓
[SLM] → clinical report / user-facing interpretation
```

The LLM teacher operates on the same affective embedding + emotion label and generates high-quality natural language outputs (diagnostic notes, affect summaries, treatment suggestions). The SLM student — a quantized Qwen2-0.5B or Phi-3-mini — is then fine-tuned

to produce the same outputs given the same inputs, using the teacher's logits and/or chain-of-thought traces as supervision.

EEG Emotion Copilot already demonstrated this direction: they took Qwen2-0.5B, applied structured pruning to reach 50% parameter reduction, then fine-tuned with LoRA on EEG-grounded emotion-to-text pairs. The LLM (full Qwen2 or GPT-4) can be used to generate the training corpus itself — synthetic but high-quality (emotion label, EEG feature summary) → (clinical text) pairs that the SLM then learns from.

The more interesting question: can the signal-level understanding be distilled too?

This is where your architecture gets genuinely novel. There's a growing body of work on distilling **biosignal understanding** — not just text generation — from LLMs into compact models:

- **EmotionKD** and **BioX-Bridge**-style frameworks (from your reading list) do exactly this: an LLM processes multimodal context (EEG features + metadata) and the distillation target is the student wearable model's affective representation, not its text output.
- The **teacher signal** in this case is the LLM's internal representation of the emotional state, which has been enriched by pretraining on vast amounts of psychology, clinical, and affective science text. This representation is richer than a simple class label.
- The student (your ECG/HRV/EDA model) learns to produce an embedding that aligns with this richer teacher representation — effectively inheriting the LLM's semantic understanding of emotional states without inheriting its parameter count.

This is essentially a **cross-modal knowledge distillation**: LLM (text-domain emotional understanding) → wearable biosignal model (signal-domain). It's architecturally analogous to CLIP-style alignment but the modalities are LLM hidden states and peripheral biosignals.

Practical constraints and what they mean for you

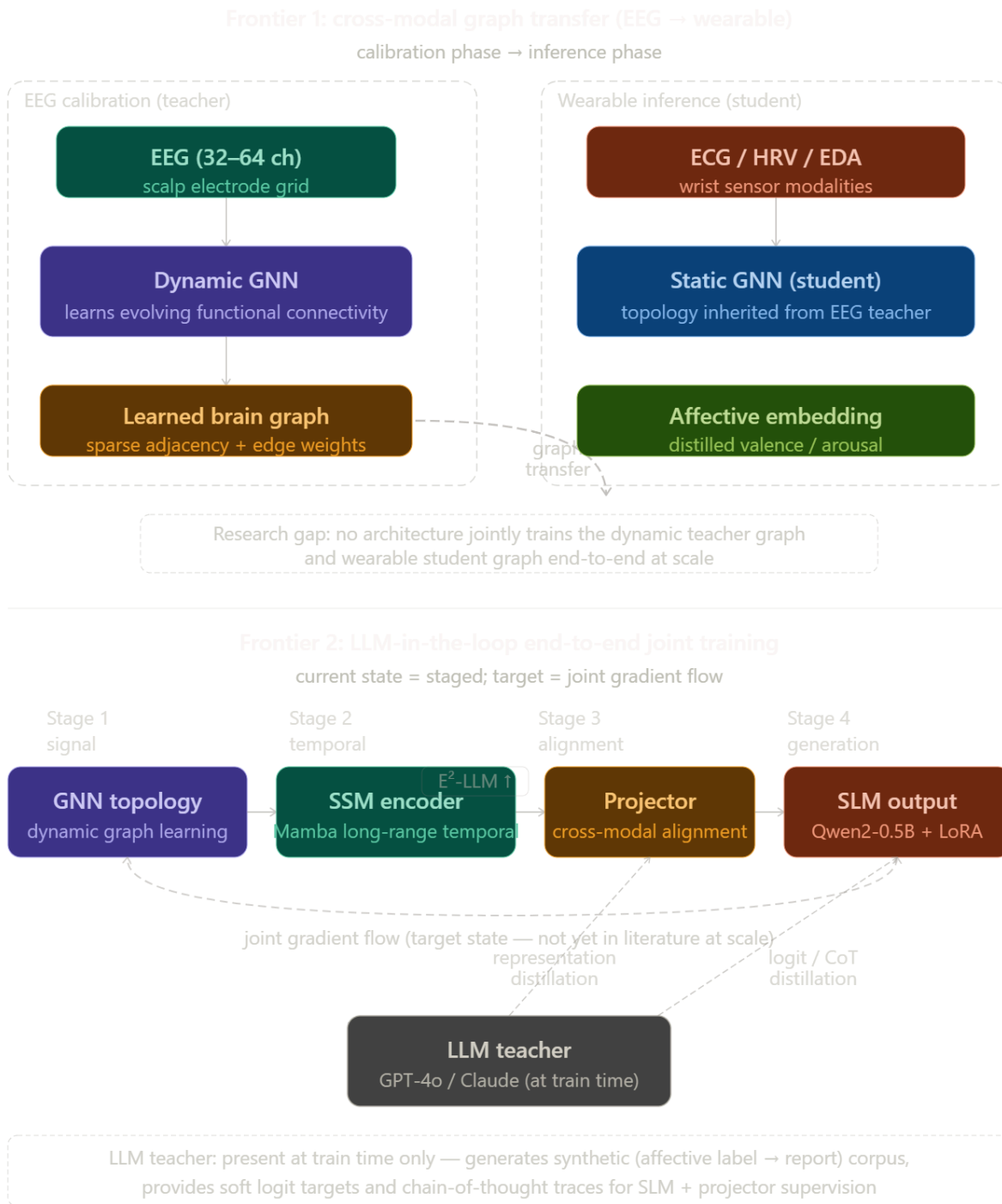
Constraint	Implication
Edge deployment (wrist sensor)	SLM must be $\leq 1\text{B}$ params, INT4/INT8 quantized; text generation may move to a phone-side model rather than the sensor itself
Privacy (on-device processing)	Favors local SLM over cloud LLM; LoRA fine-tuning is sufficient for domain adaptation

Constraint	Implication
Latency	SLM runs on output of GNN+SSM decoder, so it's on the non-critical path — latency budget is looser than for the signal backbone
Training data scarcity	LLM-generated synthetic (label → text) pairs can bootstrap the SLM without needing thousands of annotated EEG-report pairs

The distillation pipeline that makes most sense for your stack is therefore:

1. Use a frontier LLM (GPT-4o or Claude) to generate a synthetic corpus of (emotion label + EEG feature summary → clinical interpretation) pairs
2. Fine-tune Qwen2-0.5B or Phi-3-mini on this corpus using LoRA, with the LLM's output logits as soft targets
3. Quantize to INT4 for phone-side deployment
4. The GNN+SSM backbone remains the dedicated signal decoder — the SLM never sees raw EEG, only the structured output of the backbone

This architecture keeps the signal processing tight and efficient, and keeps the language generation modular and upgradeable independently. The LLM teacher is only needed at training time — inference is fully local.



Frontier 1: Cross-modal graph transfer (EEG → wearable)

What the gap actually is

A dynamic graph is one where connectivity and, in some cases, the nodes change over time — reflecting how functional brain connectivity varies during a cognitive task or in response to a stimulus. This is precisely what makes EEG-derived graphs so information-rich during calibration: the topology isn't fixed. There are not only functional differences in EEG feature

distributions but also structural differences — differences in brain functional connectivity — across subjects. Neuroscience studies confirm that functional connectivity differs significantly across different emotional states.

The problem is that this rich, emotion-specific connectivity information is currently stranded inside the EEG layer. Existing EEG+peripheral fusion work, like the approach in graph theoretical analysis of EEG functional connectivity combined with peripheral physiological signals, extracts graph features from EEG and statistical features from peripheral signals separately, then concatenates and feeds them to a classifier. The graph topology learned from EEG is never *used to structure the peripheral signal model*. The peripheral signals (ECG, EDA, HRV) are treated as independent time series, not as a graph that was informed by the brain's wiring.

What the transfer would actually look like

The core idea has a close analogue in a different domain. The **knowledge-distilled GNN for seizure detection** (EPFL, 2023) is the closest published precedent: it proposes transferring knowledge from a sophisticated seizure detector trained on a full set of EEG electrodes to learn new detectors that use significantly fewer electrodes. Both teacher and student are GNNs, since these architectures actively use connectivity information. Using as few as two channels, competitive seizure detection performance is achievable. This is intra-modal graph distillation — same signal type, fewer nodes. What your architecture needs is *cross-modal* graph distillation: EEG teacher graph → ECG/HRV/EDA student graph.

The mapping mechanism would operate as follows. During calibration, a dynamic GNN learns a subject-specific emotion-state-conditioned adjacency matrix over EEG channels — using PLV, coherence, or a learned metric. This matrix encodes which brain regions co-activate for which emotional transitions. The question then becomes: which entries in that matrix have analogues in the wearable sensor domain? ECG captures cardiac autonomic response, HRV captures parasympathetic tone, EDA captures sympathetic arousal — these are precisely the peripheral projections of the autonomic nervous system branches that the EEG functional connectivity is indexing. The transfer is not a direct adjacency copy but a *learned projection*: a small MLP trained to map EEG connectivity patterns to wearable sensor relationship weights.

The output is a personalized static graph for the wearable student — not universal (which would miss subject-specific autonomic patterns) but not re-learned from scratch on EEG each time either.

Why this doesn't exist at scale yet

Two obstacles: (1) the cross-modal topology alignment requires paired EEG+wearable training data with synchronized emotional labels — the WESAD and DEAP datasets have this, but not at the scale needed to learn a reliable mapping function. (2) There is no established loss

formulation for "make the student graph's topology reflect the teacher graph's functional meaning" — existing graph distillation work optimizes for task accuracy on the same signal type, not for semantic topology transfer across modalities.

Frontier 2: Jointly fine-tuned GNN + SSM + LLM — yes, the LLM teacher is essential

You are right to push on this. The staged training problem is real and specific.

What staged training currently looks like

The state of the art — including the very recent **E²-LLM** (January 2026) — uses a multi-stage frozen-component approach: Stage 1 trains the EEG encoder with a classification objective. Stage 2 freezes the encoder and trains the projector to align EEG representations with LLM embeddings. Stage 3 jointly fine-tunes the projector and LLM for generating natural language emotional analysis reports.

The signal encoder (which in your case would be GNN+SSM) is trained to discriminate emotions in Stage 1 without any signal from the language generation objective. Then the projector is trained without gradients flowing back into the encoder. The encoder never gets updated based on what the LLM actually needs from it. This is a fundamental information bottleneck: the encoder learns the best signal for a discrete classification head, not the best signal for rich affective reasoning.

One paper studying this explicitly found that for small-scale EEG datasets, treating the LLM as a fixed reasoning engine and focusing solely on the projection layer is the most effective strategy — full fine-tuning results in catastrophic overfitting, and even LoRA tends to degrade the LLM's pre-trained linguistic capabilities. This is a symptom of the problem, not a solution: the dataset isn't large enough to support joint training, so staged training is a workaround.

Where the LLM teacher enters, and why it's not optional

The open frontier isn't just "train everything together." The question is: *what provides the training signal that makes joint optimization converge correctly?* This is where the LLM teacher becomes load-bearing, not optional.

It plays three distinct roles simultaneously:

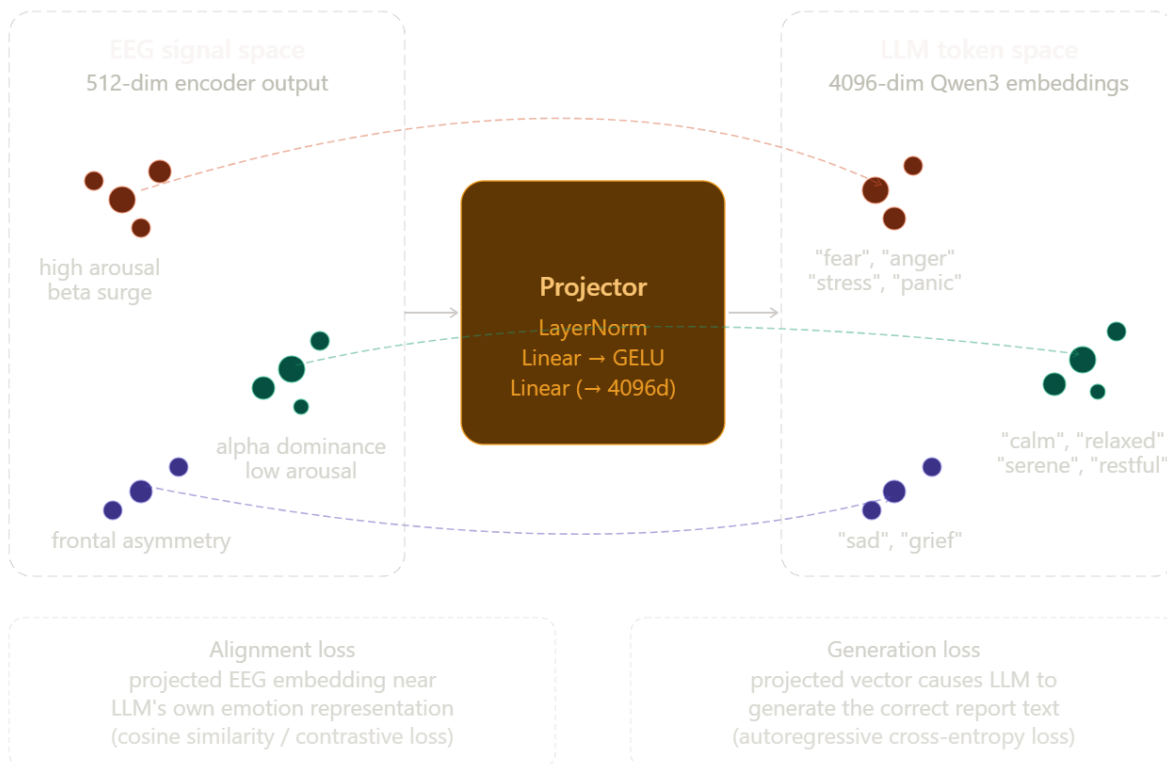
Role 1: Synthetic corpus generation. The LLM teacher generates the (affective label + EEG feature summary → clinical/interpretive report) training pairs that the SLM learns to produce. Without this, the SLM has no training data for the generation task. This role exists even in staged pipelines.

Role 2: Representation distillation target. In a jointly trained pipeline, the projector and GNN+SSM encoder receive gradient signal not just from the classification head but from alignment with the LLM's internal representation of emotional states. The LLM's hidden states encode a rich, semantically grounded understanding of affective concepts derived from pretraining on psychology, clinical, and neuroscience text. This semantic richness cannot be constructed from EEG labels alone. The GNN+SSM encoder is trained to produce embeddings that, when projected, sit near the LLM's own representation of the corresponding emotional state. This is the cross-modal CLIP-style alignment step — and it requires the LLM to have *already learned* what "high arousal, negative valence" looks like in language space before the encoder is trained.

Role 3: Chain-of-thought distillation. Conventional systems operate as closed-set classifiers that output categorical labels without interpretable rationales, lacking hierarchical cross-modal alignment mechanisms to capture both local cues and global affective patterns. The LLM teacher generates step-by-step reasoning chains — "the elevated HRV variability combined with frontal asymmetry suggests autonomic withdrawal consistent with approach-avoidance conflict" — and the SLM is trained on these traces, not just final labels. This is what allows the SLM to produce clinically meaningful output rather than templated responses.

The architecture that doesn't yet exist at scale would look like this: GNN+SSM encoder, jointly trained with a frozen LLM teacher providing representation distillation targets at the projector layer and chain-of-thought traces at the SLM output layer, with curriculum learning managing the staged warm-up problem (start frozen, gradually unfreeze from output layer inward). The LLM is present only at training time — the deployed system is GNN+SSM+projector+SLM, with the LLM having shaped every weight in the stack without appearing at inference.

This is architecturally analogous to what **NeuroLM** attempted — a universal multi-task foundation model with 1.7B parameters pre-trained on approximately 25,000 hours of EEG data, using instruction tuning to unify diverse EEG tasks within a single model — but without the cross-modal teacher-student structure that would allow the signal encoder and language generator to co-evolve. That co-evolution, scaffolded by the LLM teacher, is the unsolved problem.



Role of Projector Module

The GNN+SSM encoder produces a vector that lives in *EEG signal space* — its dimensions encode things like "channel 12 and channel 27 are strongly correlated at 8–12 Hz during this 2-second window." The LLM operates in *token embedding space* — its dimensions encode things like "this is semantically near the concept of sadness and far from the concept of excitement."

These two spaces are completely unrelated. Their dimensionalities differ (the encoder might produce a 512-dim vector; Qwen3-8B has a 4096-dim embedding space). More importantly, their *geometry* differs — nearby points in EEG embedding space are not nearby in language embedding space and vice versa. The LLM cannot interpret a raw EEG embedding any more than it can interpret a raw pixel array.

The projector is the learned bridge between these two spaces.

What it is structurally

In its simplest form the projector is a linear layer:

```
z_eeg (512-dim) → [Linear(512 → 4096)] → z_token (4096-dim)
```

In practice it is usually a small MLP with one or two hidden layers and a normalisation step:

```
z_eeg → LayerNorm → Linear → GELU → Linear → z_token
```

The output `z_token` is a vector in the exact same space as the LLM's token embeddings — the same space that the word "melancholy" or "anxious" or "calm" occupies. Once projected, the EEG representation can be inserted directly into the LLM's input sequence using special delimiter tokens (`<eeg_start>` ... `<eeg_end>`), and the LLM treats it like any other token in its context.

E²-LLM implements exactly this: EEG signals are processed by an EEG encoder, whose representations are mapped to the embedding space of a Qwen3-based LLM via a projector. Special tokens `<eeg_start>` and `<eeg_end>` demarcate EEG segments within the input sequence, enabling the LLM to generate interpretive emotional analysis.

What it learns during training

The projector is trained with two objectives simultaneously, which is what makes it non-trivial:

Alignment objective: The projected EEG embedding should be geometrically close to the LLM's own representation of the ground-truth emotion label. If the EEG segment corresponds to "high arousal, negative valence," the projected vector should sit near where the LLM internally represents that concept. This is the contrastive/cosine alignment loss.

Generation objective: The projected vector, when placed in the LLM's input context, should cause the LLM to generate the correct natural language description — the clinical report, the emotion label, the chain-of-thought trace. This is the autoregressive cross-entropy loss.

These two objectives pull in slightly different directions. The alignment loss wants the projector to be a pure translation function. The generation loss wants the projector to preserve whatever EEG features are most *useful for generating good language*, which may not be the same as the features most *similar to the LLM's emotion representations*. The tension between them is what forces the projector to learn something genuinely useful rather than collapsing to a trivial mapping.

How it fits into the three-stage training protocol

The staged training used in E²-LLM and similar architectures reveals the projector's central role: The diagram captures the key intuition: the projector is not just a dimensionality converter

— it is learning a *semantic rotation* of one space into another. Same-emotion clusters from EEG signal space (coral = high arousal, teal = calm, purple = negative valence) should land near their corresponding token clusters in LLM space after the projector transforms them.

The three failure modes that make projector design hard

Modality gap collapse. If the alignment loss dominates, the projector learns to map every EEG embedding to the centroid of the LLM's emotion concept cluster, erasing the fine-grained variation within an emotion. The result is a projector that maps "high-arousal fear" and "low-arousal apprehension" to the same point. The generation loss counteracts this by forcing the projector to preserve distinctions that matter for producing nuanced language.

Catastrophic overfitting. As noted in ablation studies, if the LLM is also unfrozen during projector training, the LLM's pre-trained language geometry collapses toward the EEG data distribution. With only a few thousand labeled EEG segments available, the LLM's billions of parameters have far more capacity than the data can fill, and it overfits immediately. This is why the staged freeze strategy exists: it is not a design preference but a dataset size constraint.

Dimensionality mismatch asymmetry. A 512→4096 projection is expanding, not compressing. Standard MLPs doing this expansion tend to produce high-variance embeddings that don't sit cleanly in the LLM's embedding manifold. LayerNorm before the linear transform is essential — it scales the EEG embedding to have the same norm distribution as the LLM's own input embeddings, which stabilises training significantly.

What this means for specific pipeline

In the GNN+SSM+Projector+SLM architecture, the projector inherits a specific set of properties from your encoder choices. The GNN output encodes relational structure — which channels are connected, with what weights — while the SSM output encodes temporal dynamics. The projector therefore receives a vector that is richer in structure than a standard CNN-based EEG encoder would produce. This is an advantage: the LLM has a natural vocabulary for relational and temporal concepts ("the prefrontal cortex showed sustained engagement while the amygdala responded transiently"), and a well-trained projector can map structured GNN+SSM representations directly onto that vocabulary in a way a flat feature vector could not.

The concrete implication is that your projector can potentially be smaller — fewer hidden units, maybe a single linear layer — because the GNN+SSM encoder is already doing the semantic

heavy lifting. The projector's job becomes primarily dimensional scaling rather than conceptual re-encoding.